Op3nSRC Technical Report

Iftekhar Ahmed, Hong Phan, Nivedha Rengaraj
Data Science, University of Helsinki

October 2026

1 Introduction

Many developers want to contribute to open source but do not know where to start. GitHub holds thousands of projects and issues, and most are a poor fit for any one person. Our target group is beginner and intermediate developers looking for a project and an issue that match their skills.

Op3nSRC is a web application that solves this in one step. A user enters a GitHub username or describes their interests. The app recommends five projects that fit them and three open issues inside each, with the shared words that explain every match. The code is in our GitHub repository.

2 Data collection

All data comes from the public GitHub REST API, collected with our own Python scripts on 30 September and 1 October 2026. The team hand-picked 219 popular, active repositories across Python, JavaScript, TypeScript, Go, Rust, Java and C#. We later dropped 17. Twelve have GitHub Issues turned off, such as Django and PostgreSQL, and five are archived or inactive. The remaining 202 repositories are the pool the app recommends from.

Data	What we kept	Why we need it
Repository info and README	description, topics, languages, stars, README text	describes each project
Issues	active since 24 June 2026, at most 100 per repository	the things we recommend
Merged pull requests	PRs that close an issue, at most 50 per repository	show who works on which project

Table 1: Data collected for each repository.

Merged pull requests were the key choice. Each one proves that a real person contributed to a real project, which gives us labels without manual work. The collector waits out rate limits and resumes where it stopped, and a full run takes about 30 minutes. The final dataset has 15,053 issues (7,453 open) and 4,003 merged pull requests from 1,843 developers.

The first run broke in three ways. A project with issues turned off ended the whole run at repository 108. The pull request endpoint ignored our date filter and paged through years of history. Twenty-eight READMEs were saved as error pages because we guessed the file name. We fixed each problem and collected the rest again.

3 Data processing

A cleaning script turns the raw JSON into three tables of repositories, issues and pull requests.

1. **Remove duplicates and bots.** We dropped accounts GitHub marks as bots, logins ending in bot or robot, and three CI accounts that post test failure reports. This removed about 280 issues and 125 pull requests, 71 of them written by the Copilot agent.

2. **Clean the text.** We stripped code, links, images, HTML, markdown and issue template text, then lowercased it.

3. **Mark beginner issues.** We matched label variants such as good first issue, e-easy and first-contribution, giving 210 beginner-friendly and 226 help wanted issues.

4. **Connect pull requests to issues.** Closing keywords like "fixes #123" show which pull request solved which issue.

The model learns from one row per developer. The input is the titles and descriptions of their merged pull requests and the titles of issues they opened. The label is the project they merged most pull requests into. Keeping projects with at least five developers gives 1,724 developers over 117 projects.

Two steps keep this honest. We removed every project and owner name from the input, since pull requests often name their own project. We also removed pull request template sentences, meaning any sentence found in three or more PRs of one project. Templates identify the project, not the person, and keeping them added about 5 points of accuracy for the wrong reason.

4 Exploratory analysis

The data is concentrated in a few ecosystems. Python, Go, Rust, TypeScript and JavaScript make up 80% of the repositories, while Java, C# and C++ have only 28 between them.

Issues are long and noisy. The median issue has 150 words, and the most common phrases are template headings such as "steps to reproduce". We therefore let TF-IDF ignore words found in more than half of the documents. Labels are inconsistent, with 3,154 different names, and only 2.6% of issues are marked beginner-friendly or help wanted. We use these labels as a small boost, not as the main signal.

Most developers leave a thin trace. Sixty-eight percent have exactly one merged pull request, and only 7% contributed to more than one project.

5 Machine learning

We framed project recommendation as supervised text classification. From what a developer has written, the model predicts which of the 117 projects they contribute to, and the app shows the five most likely ones. Each developer appears once, so a standard random train and test split is valid.

Text is turned into numbers with TF-IDF, which weights rare, specific words above common ones. We used single words, word pairs and character pieces of three to five letters, which help with code names like `strftime`.

We compared three standard models with a baseline that always guesses the most common project. The split was 80/20 with stratification, giving 1,379 training and 345 test developers. Settings were tuned with four-fold cross-validation on the training set, and the test set was used once. Our main metric is top-5 accuracy, the share of test developers whose real project is among the five we would show.

Both TF-IDF models beat the baseline by far. Sentence-BERT came last because the link between a person and a project is mostly exact library names, which a meaning-based embedding blurs.

Model	Accuracy	Top-5 accuracy	Macro F1
Most frequent project (baseline)	2%	6%	0.00
TF-IDF + Naive Bayes	75%	88%	0.68
TF-IDF + Logistic Regression	69%	84%	0.55
Sentence-BERT (all-MiniLM-L6-v2) + Logistic Regression	41%	67%	0.28

Table 2: Results on the held-out test set (345 developers, 117 projects).

Naive Bayes scored highest, while the deployed app runs Logistic Regression at 84% top-5 accuracy.

Inside each recommended project, open issues are ranked by TF-IDF cosine similarity to the user's text, with a 0.05 bonus for beginner-friendly labels. The explanation shown is the words that add most to that similarity, such as vite and frontend. Projects without open issues are never recommended.

6 The application

The app has three parts. A React and Vite website collects the input, a FastAPI server answers on one endpoint, and the ML package holds the data pipeline and the recommender.

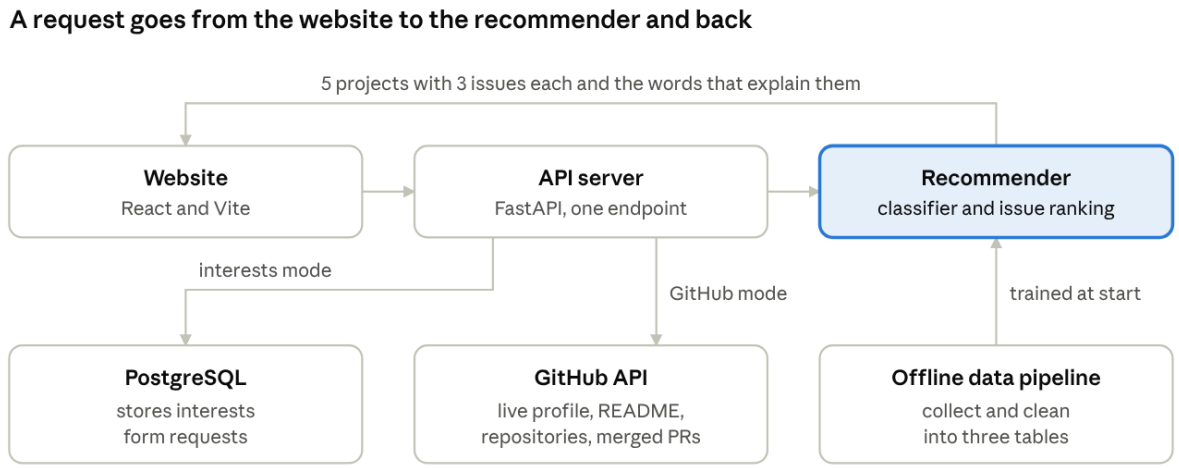


Figure 1: Op3nSRC architecture. The website, the API and the recommender, with their data sources.

When the server starts, it trains the classifier on all 1,724 developers. In GitHub mode the server fetches the user's profile live. It needs a personal README, then reads up to 30 of their own repositories, their languages and their merged pull requests, skipping forks. In interests mode the user picks a language and describes what they like, and the request is stored in PostgreSQL.

7 Reflection on the plan

Our canvas planned an NLP system that ranks single issues for a developer by text similarity, with TF-IDF as baseline, Sentence-BERT as the main model, optional clustering and ranking metrics such as NDCG.

What worked was the data idea. Using merged pull requests as labels gave us real ground truth for free, TF-IDF proved strong, and the cleaning was reproducible from the start.

What did not work was the first evaluation. We ranked issues with a time-based test, building profiles only from activity before a cut-off date. A solved issue landed in the top ten for 28% of developers, but only 88 developers could be scored, so the results were noisy and hard to explain. We switched to classifying developers into projects, which allows a standard split and clearer results. Sentence-BERT never beat TF-IDF, and we dropped clustering because our labels made supervised learning the better fit.

The product changed as well. We moved from recommending issues to recommending projects with issues inside, since people choose a project first. We also made the user profile leaner.

If we started again, we would run a small pilot collection first to find API problems early. We would also collect the training developers' own repositories, so the training text looks like what the app sees for a new user.

8 Limitations and future work

The biggest limitation is the gap between training and use. The model learns from pull request text, but a new user's profile is mostly their README, repositories and languages. Live recommendations are therefore looser than the test score, especially for users without merged pull requests.

The pool is fixed at 116 recommendable projects from an October 2026 snapshot, so some issues may already be closed. The issue suggestions were not tested with real users, and developers outside the five main languages get weaker results. GitHub profiles are never stored, but interests form entries are, and users should be told so.

As next steps we would train on developers' own repositories, widen the project list and refresh the data regularly. We would also switch the app to Naive Bayes, test the issue suggestions with real users and add a privacy note to the interests form.

9 Learning outcomes

The project took us through the whole data life cycle, from raw API responses to a working app. The data work took more time than the modelling, and most of the quality came from it.

We learned that collecting data at scale is mostly about edge cases, since rate limits, disabled features and missing files all broke our first run. We learned to check data before trusting it. Bots, templates and project names in the text would each have made our results look better than they were.

We also learned that how a problem is framed decides how it can be evaluated, and that a newer model is not always the better one for the data at hand. Keeping shared notes and a decision log helped all three of us understand why each choice was made.